

DAY-20 — Basic Security Hardening

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **basic security hardening**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Basic Security Hardening is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: updates, accounts, SSH hygiene, services, permissions, backups, firewall basics, baseline.

Reference	What to remember
<code>apt update</code>	refresh package metadata
<code>apt upgrade</code>	apply available updates
<code>ss -tuln</code>	review listeners
<code>systemctl</code>	review services
<code>ufw</code>	host firewall interface
<code>chmod</code>	review file access

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. updates

Applying updates promptly patches known vulnerabilities before they can be exploited — ``apt update`` refreshes the list of available packages, ``apt upgrade`` actually installs newer versions. A large share of real-world breaches exploit vulnerabilities for which a patch already existed but simply hadn't been applied yet.

2. accounts

Hardening accounts means removing unused or default accounts, disabling direct root login where possible, and confirming every remaining account is actually needed and has appropriately scoped access — directly

applying the least-privilege principle from Day 6.

3. SSH hygiene

SSH hygiene includes disabling password authentication in favor of key-based login, disabling direct root SSH login, and optionally changing the default port to reduce automated scanning noise. Since SSH is often the primary remote-administration door into a server, it's a high-value target to lock down carefully.

4. services

Reviewing running services and disabling anything unnecessary shrinks the attack surface (Day 14) — every extra service listening on the network is one more potential entry point that has to be kept patched and monitored. `systemctl --type=service --state=running` shows what's currently active.

5. permissions

Reviewing file and directory permissions (Day 4) as part of hardening means checking for overly permissive files — world-writable configs, keys readable by everyone — which are easy wins for both attackers and auditors to find.

6. backups

Backups ensure that a security incident, hardware failure, or ransomware attack doesn't mean permanent data loss. A backup strategy is only as good as its last successful test-restore — an untested backup is an assumption, not a guarantee.

7. firewall basics

A host-based firewall like `ufw` (an easier front-end for `iptables/nftables`) controls which network traffic is allowed to or from the machine, letting you close off everything except the ports you specifically intend to expose — a practical, direct application of least privilege at the network level.

8. baseline

A security baseline is a documented, known-good configuration — which services run, which ports are open, what the normal user list looks like — that you can compare against later to quickly spot unauthorized changes. Establishing a baseline early is what makes "has anything changed?" answerable instead of a guess.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
sudo apt update
apt list --upgradable 2>/dev/null | head
ss -tuln
systemctl --type=service --state=running 2>/dev/null | head
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does updates solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain basic security hardening in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-20, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-20 — Basic Security Hardening

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `apt update` — refresh package metadata
- `apt upgrade` — apply available updates
- `ss -tln` — review listeners
- `systemctl` — review services
- `ufw` — host firewall interface
- `chmod` — review file access

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.